Solucionario Docente: JDBC, CRUD y Transacciones ACID

Soluciones de referencia completas y documentadas para los ejercicios del Bloque 1

MÓDULO	UNIDAD	BLOQUE	PROFESOR
Acceso a Datos (0486)	UT2: JDBC y MySQL	B1: CRUD y Transacciones	Sergio Capdevila Díez

Resultados de Aprendizaje: RA2 **Criterios de Evaluación:** `CE2.a` , `CE2.c` , `CE2.d` , `CE2.g`

Ejercicio 2.1: CLI Interactivo de Inventario con PreparedStatement

45 MIN

Desarrolla una aplicación de consola en Java que se conecte a MySQL 8 y permita: (1) Añadir un artículo con nombre, precio y stock devolviendo el ID autogenerado, (2) Listar todos los artículos con formato de tabla ASCII, (3) Actualizar el stock de un producto buscando por ID.

🔑 SOLUCIÓN COMPLETA Y CÓDIGO FUNCIONAL:

```
package es.cifpaviles.dam2.ad.ut2.b1;

import java.math.BigDecimal;
import java.sql.*;

public class InventarioCLI {
    private static final String URL = "jdbc:mysql://localhost:3306/dam2_db?useSSL=false&allowPublicKeyRetrieval=true";
    private static final String USER = "dam2_user";
    private static final String PASS = "dam2_pass";

    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(URL, USER, PASS)) {
            System.out.println("Conexión exitosa. Insertando producto de prueba...");
            long id = insertarProducto(conn, "Teclado Mecánico RGB", new BigDecimal("79.95"), 12);
            System.out.println("Producto insertado con ID: " + id);

            System.out.println("\nListando inventario:");
            listarProductos(conn);

            System.out.println("\nActualizando stock a 20 unidades...");
            actualizarStock(conn, id, 20);
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    public static long insertarProducto(Connection conn, String nombre, BigDecimal precio, int stock) throws SQLException {
        String sql = "INSERT INTO productos (nombre, precio, stock) VALUES (?, ?, ?)";
        try (PreparedStatement ps = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
            ps.setString(1, nombre);
            ps.setBigDecimal(2, precio);
            ps.setInt(3, stock);
            ps.executeUpdate();
            try (ResultSet rs = ps.getGeneratedKeys()) {
                if (rs.next()) return rs.getLong(1);
            }
        }
        return -1;
    }

    public static void listarProductos(Connection conn) throws SQLException {
        String sql = "SELECT id_producto, nombre, precio, stock FROM productos ORDER BY id_producto";
        try (PreparedStatement ps = conn.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            System.out.printf("%-6s | %-25s | %-9s | %-5s\n", "ID", "NOMBRE", "PRECIO", "STOCK");
            System.out.println("-------+---------------------------+-----------+------");
            while (rs.next()) {
                System.out.printf("%-6d | %-25s | %-9.2f | %-5d\n",
                    rs.getLong("id_producto"), rs.getString("nombre"),
                    rs.getBigDecimal("precio"), rs.getInt("stock"));
            }
        }
    }

    public static void actualizarStock(Connection conn, long id, int nuevoStock) throws SQLException {
        String sql = "UPDATE productos SET stock = ? WHERE id_producto = ?";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, nuevoStock);
            ps.setLong(2, id);
            int filas = ps.executeUpdate();
            System.out.println("Filas actualizadas: " + filas);
```

```
        }
    }
}
```

Ejercicio 2.2: Transacción Bancaria con Savepoints y Auditoría de Fallos

55 MIN

Crea una tabla `movimientos_auditoria (id BIGINT AUTO_INCREMENT PRIMARY KEY, operacion VARCHAR(100), estado VARCHAR(20), fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP)` (motor InnoDB). Desarrolla una operación transaccional que intente transferir dinero. Si el saldo no alcanza, la transacción debe hacer rollback a un Savepoint para no mover dinero, pero debe registrar en la tabla de auditoría el intento fallido y confirmarlo.

🔑 SOLUCIÓN COMPLETA Y CÓDIGO FUNCIONAL:

```
package es.cifpaviles.dam2.ad.ut2.b1;

import java.math.BigDecimal;
import java.sql.*;

public class TransaccionConAuditoria {
    private static final String URL = "jdbc:mysql://localhost:3306/dam2_db?useSSL=false&allowPublicKeyRetrieval=true";
    private static final String USER = "dam2_user";
    private static final String PASS = "dam2_pass";

    public static void main(String[] args) {
        ejecutarTransferenciaSegura(1L, 2L, new BigDecimal("5000.00")); // Provocará saldo insuficiente
    }

    public static void ejecutarTransferenciaSegura(long origen, long destino, BigDecimal importe) {
        String sqlDebitar = "UPDATE cuentas_bancarias SET saldo = saldo - ? WHERE id = ? AND saldo >= ?";
        String sqlAcreditar = "UPDATE cuentas_bancarias SET saldo = saldo + ? WHERE id = ?";
        String sqlAuditoria = "INSERT INTO movimientos_auditoria (operacion, estado, fecha) "
                            + "VALUES (?, ?, CURRENT_TIMESTAMP)";

        try (Connection conn = DriverManager.getConnection(URL, USER, PASS)) {
            conn.setAutoCommit(false);
            Savepoint spInicio = conn.setSavepoint("PuntoInicial");

            try {
                // Paso 1: Intentar debitar
                try (PreparedStatement ps = conn.prepareStatement(sqlDebitar)) {
                    ps.setBigDecimal(1, importe);
                    ps.setLong(2, origen);
                    ps.setBigDecimal(3, importe);
                    if (ps.executeUpdate() == 0) {
                        throw new IllegalStateException("Saldo insuficiente en cuenta origen.");
                    }
                }

                // Paso 2: Acreditar
                try (PreparedStatement ps = conn.prepareStatement(sqlAcreditar)) {
                    ps.setBigDecimal(1, importe);
                    ps.setLong(2, destino);
                    if (ps.executeUpdate() == 0) {
                        throw new IllegalStateException("Cuenta destino inexistente.");
                    }
                }

                // Paso 3: Registrar éxito
                try (PreparedStatement psAud = conn.prepareStatement(sqlAuditoria)) {
                    psAud.setString(1, "Transferencia de " + importe + " € de cuenta " + origen + " a " + destino);
                    psAud.setString(2, "COMPLETADA");
                    psAud.executeUpdate();
                }

                conn.commit();
                System.out.println("[OK] Operación confirmada con éxito.");

            } catch (Exception e) {
                System.err.println("[AVISO] Fallo en la transferencia: " + e.getMessage());
                // Deshacer debitos/créditos hasta el Savepoint
                conn.rollback(spInicio);
```

```
                // Registrar auditoría de fallo en una nueva transacción limpia
                try (PreparedStatement psAud = conn.prepareStatement(sqlAuditoria)) {
                    psAud.setString(1, "Transferencia fallida de " + importe + " €: " + e.getMessage());
                    psAud.setString(2, "RECHAZADA");
                    psAud.executeUpdate();
                }
                conn.commit(); // Confirmar registro de auditoría
                System.out.println("[AUDITORÍA] Intento fallido registrado en el histórico.");
            } finally {
                conn.setAutoCommit(true);
            }
        } catch (SQLException ex) {
            ex.printStackTrace();
        }
    }
}
```